Project Proposal: Application for Learning Recursion

Julia Nething (Solo Project)
nething@gatech.edu

1 INTRODUCTION

Recursion is a difficult topic for novice programmers to learn. It is also a difficult topic to teach. The default recommendation to students is to practice, but it is difficult to practice a concept that students do not fundamentally understand. Students then feel frustrated and conclude that they do not have the talent or skills to be successful in this field.

As a teacher for novice adult programmers, when a student is struggling with recursion I try to walk through a problem with them or I recommend a variety of resources including videos, problem banks, and blogs. There are many scattered resources available on the internet, but there is no comprehensive platform for demonstrating recursion in different ways. There has also been no widespread research done on which strategies are most effective for students.

It would be helpful to students if there was a platform that combined many of these strategies and allowed the students to choose the strategy resonated the most with them, along with dedicated practice problems so they can test their knowledge. For this project, I propose to build a web application that combines several techniques that I read about during the research phase of this course.

2 RELATED WORK

When I was researching how recursion is taught at the undergraduate level, I found that professors are divided on their strategies.

Some choose to teach recursion at a high level using games, ignoring code completely, and then expect students to transfer that knowledge to code (Chaffin et al., 2009). One example of that is the use of Cargo-Bot at the University of Texas at Austin. In this example, students played a game where they had to move boxes using recursion from an original configuration to a goal configuration (Lee et al., 2014). There was no code involved, but students were expected to generalize the knowledge the gained in the game to code later.

Some went a step further by combing real world examples with code by having students track values during the examples. For example, in one study stu-

dent volunteers gave a live demonstration of the Tower of Hanoi problem while students in the audience took notes on the values as the problem progressed (Benander and Benander, 2008).

Others show structural recursion, such as a bulls-eye or a tree diagram, and the corresponding code or pseudo-code (Kruse, 1982). One professor even argued that structural recursion should be taught before arrays so that students are not as intimidated (Bruce, Danyluk, and Murtagh, 2005). On a similar note, some showed recursion through visuals such as fractals or geometric shapes (Elenbogen and O'Kennon, 1988) (Gordon, 2006).

Some professors took a more abstract approach and taught recursion using mathematical induction (Ford, 1984) (Wu, Dale, and Bethel, 1998).

A common strategy was to use a recursive graph or similar visualization to show how recursive code is called by the interpreter (Hsin, 2008). At Carnegie Mellon University, professors instructed students to trace code through recursive graphs, fix broken recursive graphs, and build their own recursive graphs (AlZoubi et al., 2015).

Most of the research that I read did not effectively test these strategies. Almost all used very small sample sizes and their experiment was not reproduced. Therefore, it is difficult to tell which strategies are the most effective. In this project, students will be able to choose which strategies they try so they can find the ones that resonate with them the most.

In industry, recursion is taught via YouTube videos and blogs. There are also practice problems on websites such as Leetcode, Interview Cake, and Exercism.io, but there is no website specifically designed for dedicated practice of recursion (these websites feature other types of problems also). With so many resources available, it is difficult for students to decide which resource to use. If there was a curated list of resources that may be more helpful for students.

3 PROPOSED WORK

3.1 Overall Application

I am proposing to build a web application that will help teach students recursion. There are four pages to this application: the user account page, the practice page, the high level walkthroughs page, and the real code walkthroughs page.

The application will be written with a React frontend, Node/Express backend, and MySQL database. I will use Material-UI, a React UI framework, to make design easier.

Users will access the application by visiting a website (it will be deployed) and log in through their GitHub account (Passport.js authentication with GitHub). On their first login, they will be prompted to take a pre test to assess their baseline understanding of recursion.

3.2 User Account Page

The user account page will be very minimal and only show key items like account creation date and GitHub handle.

3.3 Practice Page

The practice page is where students will go to practice various recursive problems and get immediate feedback. This combines the pedagogical goals of both feedback and dedicated practice. I do not plan to create problems and tests but rather curate a list of recursive problems and tests from pre-existing sites such as Leetcode. Ideally, I will be able to integrate the user's Leetcode account so that when they submit their answer to Leetcode it updates their practice page accordingly to show that the problem is complete.

If this is too difficult to accomplish, one backup plan is to create a command-line interface (CLI) like exercism.io does. On that website, students submit their code via the CLI and it updates their exercism.io account. In my research I found that there is already a CLI for submitting to Leetcode, so it doesn't sound too difficult to combine those so that this project's CLI calls Leetcode's CLI and updates both.

A final backup plan is to simply show a link to the recursive problems, and then students can manually mark the problems as complete on this project website.

For each of the problems displayed, they will be tagged for type of recursive problem and difficulty level. There will also be a discussion board on the page so users can share resources and collaborate. The different types of recursive problems will be shuffled and the type will not be showed to the student until after completion in order to implement interleaving, where students or shown different types of problems in a random order.

One possible problem with this plan is if there are legal issues with using the Leetcode problems. In this situation, I will either find a question bank that I can legally use or I will write questions myself. This would not be ideal because then students would not get immediate feedback via the Leetcode testing framework.

3.4 High Level Walkthroughs Page

The High Level Walkthroughs page is where students will go to review recursion at a high level. There will be at least two examples of recursion on this page; options for problems to demonstrate include Towers of Hanoi, Missionaries and Cannibals, Telephone Game, a Tree diagram, or Robot Paths.

For each example, there will be a video demonstrating the problem and a corresponding exercise/quiz that students need to complete. The exercise will include questions about what the values were at different times, how long each recursive call was, and did this improve their understanding of recursion. This will help students engage in active learning instead of passively watching a video. There will also be a discussion board on this page so that students can answer each others' questions and collaborate on understanding the topic.

3.5 Real Code Walkthroughs Page

The Real Code Walkthroughs page is where students will go to review recursion at a code level. There will be at least two examples of recursion on this page; options will be taken from the practice problems.

For each example, there will be a video demonstrating how to solve the problem using code, preferably using more than one method such as the code substitution strategy. There will also be a corresponding exercise/quiz that students need to complete. The exercise will include questions about what the values were at different times, when did each recursive call end, what steps did we employ to solve the problem, and did this improve their understanding of recursion. This will help students engage in active learning. There will also be a discussion board so students can collaborate and answer each others' questions.

If there is time, I also want to make an interactive component where students can click through pre-written code along with the video. This may not be feasible and is a nice-to-have feature (this is labeled "Code Substitution component" in the task list).

3.6 Evaluation

The goal is that users will be able to see their own improvement in recursion over time. If users do complete several exercises and practice problems, it will be interesting to see how their pre and post quizzes reflect that knowledge and if certain resources are more or less beneficial than others.

4 DELIVERABLES

4.1 Intermediate Milestone 1

For the first intermediate milestone, I will be showing the basic web application and practice page. Users will be able to log in via GitHub and track their progress on the practice problems. The practice problems will also be tagged with recursion problem type and difficulty level. I will not be writing the practice problems and tests myself but rather curating a list of pre-existing problems and allowing students to track their progress on one page.

4.2 Intermediate Milestone 2

For the second intermediate milestone, I will be showing the High Level Walkthroughs page. This page will allow students to watch videos and answer questions about high-level recursive problems, such as the Tower of Hanoi. The full page will be complete at this time. I will also show the basic infrastructure for the Real Code Walkthroughs page. That page will not be complete by the time of the milestone but will have the SQL tables, endpoints, and React components in place. The Real Code Walkthroughs page will allow students to watch videos and answer questions about specific example recursive problems using JavaScript.

4.3 Final Project

For the final project deliverable, I will put the final touches on all the pages, add additional content (videos/quizzes), and deploy the application. This will include the link to a deployed application, code, videos, pre and post tests, and design documents.

5 TASK LIST

The task list is being forced to the bottom of the document due to its height, please scroll down to find it.

6 REFERENCES

[1] AlZoubi, Omar, Fossati, Davide, Di Eugenio, Barbara, Green, Nick, Alizadeh, Mehrdad, and Harsley, Rachel (2015). "A Hybrid Model for Teaching Recursion". In: *Proceedings of the 16th Annual Conference on Information Technology Education*. SIGITE '15. Chicago, Illinois, USA: Association for Computing Machinery, pp. 65–70. ISBN: 9781450338356. DOI: 10.1145/2808006.2808030. URL: https://doi.org/10.1145/2808006.2808030.

[2] Benander, Alan C. and Benander, Barbara A. (Winter 2008). "Student Monks - Teaching Recursion in an IS or CS Programming Course Using the Towers of Hanoi". English. In: *Journal of Information Systems Education* 19.4. Copyright - Copyright EDSIG Winter 2008; Document feature - ; Tables; Diagrams; Graphs; Last updated - 2020-11-17, pp. 455–467. URL: https://go.openathens.net/redirector/gatech.edu?url=https://search-proquest-com.eu1.proxy.openathens.net/scholarly-journals/student-monks-teaching-recursion-is-cs/docview/200157798/se-2?accountid=11107.

[3] Bruce, Kim B, Danyluk, Andrea, and Murtagh, Thomas (2005). "Why structural recursion should be taught before arrays in CS 1". In: *SIGCSE bulletin.* 37.1, pp. 246–250. ISSN: 0097-8418.

[4] Chaffin, Amanda, Doran, Katelyn, Hicks, Drew, and Barnes, Tiffany (2009). "Experimental Evaluation of Teaching Recursion in a Video Game". In: *Proceedings of the 2009 ACM SIGGRAPH Symposium on Video Games*. New York, NY, USA: Association for Computing Machinery, pp. 79–86. ISBN: 9781605585147. URL: https://doi.org/10.1145/1581073.1581086.

[5] Elenbogen, Bruce S. and O'Kennon, Martha R. (Feb. 1988). "Teaching Recursion Using Fractals in Prolog". In: *SIGCSE Bull.* 20.1, pp. 263–266. ISSN: 0097-8418. DOI: 10.1145/52965.53029. URL: https://doi.org/10.1145/52965.53029.

[6] Ford, Gary (Jan. 1984). "An Implementation-Independent Approach to Teaching Recursion". In: *SIGCSE Bull.* 16.1, pp. 213–216. ISSN: 0097-8418. DOI: 10.1145/952980.808655. URL: https://doi.org/10.1145/952980.808655.

[7] Gordon, Aaron (Oct. 2006). "Teaching Recursion Using Recursively-Generated Geometric Designs". In: *J. Comput. Sci. Coll.* 22.1, pp. 124–130. ISSN: 1937-4771.

[8] Hsin, Wen-Jung (Apr. 2008). "Teaching Recursion Using Recursion Graphs". In: 23.4, pp. 217–222. ISSN: 1937-4771.

[9] Kruse, Robert L. (Feb. 1982). "On Teaching Recursion". In: *SIGCSE Bull.* 14.1, pp. 92–96. ISSN: 0097-8418. DOI: 10.1145/953051.801346. URL: https://doi.org/10.1145/953051.801346.

[10] Lee, Elynn, Shan, Victoria, Beth, Bradley, and Lin, Calvin (2014). "A Structured Approach to Teaching Recursion Using Cargo-Bot". In: *Proceedings of the Tenth Annual Conference on International Computing Education Research*. ICER '14. Glasgow, Scotland, United Kingdom: Association for Computing Machinery, pp. 59–66. ISBN: 9781450327558. DOI: 10.1145/2632320.2632356. URL: https://doi.org/10.1145/2632320.2632356.

[11] Wu, Cheng-Chih, Dale, Nell B., and Bethel, Lowell J. (Mar. 1998). "Conceptual Models and Cognitive Learning Styles in Teaching Recursion". In: *SIGCSE Bull.* 30.1, pp. 292–296. ISSN: 0097-8418. DOI: 10.1145/274790.274315. URL: https://doi.org/10.1145/274790.274315.

Week #	Task #	Task Description	Est. Time (Hours)	Total Hours
8	1	Set up basic React app with Webpack and Babel	2	**102.5**
8	2	Add Node/Express backend	2	
8	3	Connect MySQL database	2	
8	4	Add authentication with Passport.js and GitHub	2	
8	5	Style/design basic landing page and user account page	4	
9	6	Plan out data design and page design for the Practice page	2	
9	7	Create MySQL tables and data design for the Practice page	2	
9	8	Create Node/Express endpoints for the Practice page	2	
9	9	Create React components that call backend endpoints for the Practice page	3	
9	10	Begin Leetcode integration (or similar integration if that does not work)	5	
10	11	Continue Leetcode integration; add or switch to CLI model like Exercism.io if there are issues or if it is easier	5	
10	12	Tag practice problems with appropriate type and difficulty and tie up loose ends	2	
10	13	Integrate discussion board to Practice page	3	
10	14	Prepare Intermediate Milestone	1.5	
INTERMEDIATE MILESTONE 1 DUE				
11	15	Plan out data design and page design for the High-Level Walkthroughs (HLW) page	2	
11	16	Create MySQL tables and data design for the HLW page	2	
11	17	Create Node/Express endpoints for the HLW page	2	
11	18	Create React components that call backend endpoints for the HLW page - Video and Exercise components	5	
12	19	Integrate discussion board to HLW page (should be easier this time)	2	
12	20	Create at least one video example and quiz content for HLW page - Tower of Hanoi, Telephone, Tree, Robot Paths	4	
12	21	Create another video example and quiz content for HLW page	4	
13	22	Plan out data/page design for the Real Code Walkthroughs (RCW) page	2	
13	23	Create MySQL tables and data design for the RCW page	2	
13	24	Create Node/Express endpoints for the RCW page	2	
13	25	Create React components that call backend endpoints for the RCW page - Video, Exercise, and Code Substitution components	5	
13	26	Prepare Intermediate Milestone 2	1.5	
INTERMEDIATE MILESTONE 2 DUE				
14	27	Integrate discussion board to the RCRW page	2	
14	28	Create at least one video example and quiz content for RCRW page - fibonacci, maybe pick from the Leetcode easy problems	4	
14	29	Create another video example and quiz content for RCRW page	4	
15	30	Create pre and post testing for all pages	4	
15	31	Deployment	4	
15	32	Final bug fixes	4	
16	33	Prepare Project Presentation	3	
16	34	Prepare Project Paper	5	
16	35	Prepare Final Project Deliverable	1.5	
FINAL PROJECT DUE				